"CLOUD-NATIVE OBSERVABILITY PLATFORM ON AMAZON ELASTIC KUBERNETES SERVICE (EKS)"

A

Major Project Report

Submitted

In partial fulfilment

For the award of the degree of

Bachelor of Technology

In Department of Computer Science Engineering

Project Coordinator:
Prof. (Dr.) Om Prakash Sharma

Submitted By:
Abhay Kumar Saini (0201220001)

Project Guide:
Prof. (Dr.) Om Prakash Sharma
Professor, Faculty of Engineering and Technology

Faculty of Engineering & Technology

Department of Computer Science and Engineering

Jagan Nath University, Jaipur

Session 2022-26

Jagannath University
Jaipur | Rajasthan

Faculty of Engineering & Technology

Department of Computer Science and Engineering

CERTIFICATE

This is to certify that the major project report entitled **"Cloud-Native Observability Platform on Amazon Elastic Kubernetes Service (EKS)"** submitted by Abhay Kumar Saini for the award of degree of Bachelor of Technology in the department of Computer Science and Engineering of Jagannath University Jaipur is record of authentic work carried out by him under my guidance.

The matter embodies in this project report is the original work of the candidate and has not been submitted for the award of any other degree. It is further certified that he has worked with me for the required period in the Department of Computer Science Engineering, Faculty of Engineering and Technology of Jagannath University Jaipur.

Project Guide
Prof. (Dr.) Om Prakash Sharma
Professor, Faculty of Engineering and Technology

Signature ……

Project Coordinator
Prof. (Dr.) Om Prakash Sharma

Signature ……

Jagannath University
Jaipur | Rajasthan

Faculty of Engineering & Technology

Department of Computer Science and Engineering

CANDIDATE DECLARATION

I hereby declare that the major project titled **"Cloud-Native Observability Platform on Amazon Elastic Kubernetes Service (EKS)"** submitted for the partial fulfillment of the requirements for the Bachelor of Computer Science and Engineering at Jagannath University is my original work and has not been submitted previously at any other university or institution for any other degree or diploma.

I further declare that the work presented in this major project report is the result of my own investigations, except where explicitly stated otherwise. Any references to the work of other researchers have been duly acknowledged.

I also declare that I have adhered to the ethical guidelines and research standards set by Faculty of Engineering and technology of Jagannath University and have obtained all necessary permissions for the use of data, software, and other resources.

I extend my gratitude to Project Coordinator and Guide Prof. (Dr.) Om Prakash Sharma for their invaluable guidance and support throughout the course of this major project.

Abhay Kumar Saini
Enrollment Number: 0201220001

Signature

Jagannath University
Jaipur | Rajasthan

Faculty of Engineering & Technology

Department of Computer Science and Engineering

ACKNOWLEDGEMENT

I/We would like to express my/our sincere gratitude to all those who have supported and guided me/us throughout the course of this project.

First and foremost, I/We am/are deeply indebted to Project Coordinator and Guide Prof. (Dr.) Om Prakash Sharma, for their invaluable guidance, encouragement, and insightful feedback. Their expertise and support were crucial in shaping the direction and outcome of this project.

My heartfelt thanks go to my peer and team member for his collaboration, support, and the stimulating discussions that enriched this project.

I/We am/are grateful to Faculty of Engineering and Jagannath University for providing us necessary resources and facilities that enabled me/us to carry out this project. Additionally, I/We acknowledge the use of the following tools and services: Prometheus, Grafana, and AWS which were instrumental in the development of the Cloud-Native Observability Platform on Amazon Elastic Kubernetes Service (EKS).

Finally, I/We extend my/our deepest gratitude to my/our family and friends for their unwavering support, patience, and encouragement throughout my/our academic journey.

Abhay Kumar Saini
Enrollment Number: 0201220001

Abstract

The need for a solid observability mechanism has grown more important than ever for modern cloud-based applications deployed at scale, in order to provide reliable applications, which perform well and can be quickly fixed in case of a fault being observed. Traditional monitoring tries to approach the problem by manually deploying monitoring agents, depending on the events is insufficient in a dynamic containerised environment where pods/services are constantly created and removed by an orchestration system.

This project involves the design, implementation, and testing of a production grade Cloud-Native Observability Platform in Amazon Elastic Kubernetes Service (EKS). The platform contains two microservices, one Python FastAPI backend and a statically-served website hosted by NGINX, deployed in containers, along with a full observability stack based on Prometheus for time series metrics collection, Grafana for visualisation, and Alertmanager for intelligent notification routing.

The entire architecture is provisioned using Terraform, and divided into two execution layers. The first layer installs the Amazon Virtual Private Cloud, subnets, Internet and NAT gateways, the Elastic Kubernetes Service (EKS) cluster, Identity and Access Management roles, repositories in the Amazon Elastic Container Registry, and finally, a Bastion Host for administrative access. The second Terraform layer deploys NGINX Ingress Controller and the entire “kube-prometheus-stack” Helm release (including Blackbox Exporter for external availability monitoring).

The deployment is fully automated with a GitHub based CI/CD pipeline that applies immutable Helm upgrades to every change/ deployment to the main branch using an idempotent model, utenscious Docker image to Amazon ECR with OpenID Connect based keyless authentication to AWS. Custom PrometheusRule resources set alerting conditions on a variety of metrics including high error rates, high latency, service unavailable and external probe failure, and notifications are sent by SMTP email.

However, the project highlights how infrastructure-as-code and GitOps concepts can be integrated with industry-standard observability tools to create a system that is reproducible, traceable and maintainable while preserving the same workflow and level of automation that this code enables. The key outcomes are the seamless deployment of the Golden Signals dashboard, the successful validation of an end-to-end alerting system and the detailed documentation of more than a hundred engineering challenges that arose and were overcome in the process.

Keywords: Kubernetes, Amazon EKS, Observability, Prometheus, Grafana, Alertmanager, Terraform, CI/CD, GitHub Actions, Cloud-Native, Infrastructure as Code, Helm, Microservices, FastAPI, NGINX, Docker.

Table of Contents

Table of Figures

Chapter 1: Introduction

1.1 Background

In a decade the software industry has made a tremendous change in architecture. Readily-mutable applications running on distributed physical servers are now increasingly broken down into collections of loosely coupled microservices, packaged into a microservice as a container, orchestrated and managed by an orchestration platform [1]. Offering these features, Kubernetes has become the de facto standard for orchestrating containers: automated scheduling, self-healing, horizontal scaling, declarative configuration management. Cloud providers have been pushing the shift to Kubernetes by introduced managed Kubernetes services, such as Amazon Elastic Kubernetes Service (EKS) [2] which is widely used in production arguably more than its competitors.

However, as application architectures become more distributed, the problem of being able to understand the behaviour of the system at run-time is correspondingly growing. In microservices, a single request may pass through dozens of instances of microservices, network hops and data stores. If the problem occurs, without in-depth runtime telemetry, you can't determine the source. Observability, borrowed from the field of control theory and transferred to software systems, is a way of determining aspects of a system's state from system metrics, logs, and traces, thus solving this challenge [3].

Over the years, observability tooling has come of age in the cloud-native ecosystem [4]. The Cloud Native Computing Foundation (CNCF) is an industrial consortium focused on developing open-source technologies for cloud-native applications, including Prometheus, which is a graduated project.Prometheus, a graduated project of the Cloud Native Computing Foundation (CNCF), is considered the standard time-series metrics collection system for a Kubernetes environment [5]. Grafana is a flexible dashboard driven visualisation over Prometheus data. Alertmanager is in charge of alert routing, grouping, deduplication and delivery. These tools together make up role-playing the observability stacks that organisations, from infancy to hyperscale, deploy.

Even with these tools available, creating a production ready observability platform is not a trivial task [6]. The challenge isn't the tools, it's how we get them working together: getting service discovery to figure out how to auto-detect the new scrape targets, keeping alert rules from triggering too many times or too infrequently, creating a secure way to expose these dashboards (ingress layer), and making sure the entire system could be reproduced by anyone from a set of version-controlled configuration files. This project seeks to tackle just these issues.

1.2 Problem Definition

Because a Kubernetes deployment doesn't have a well-defined code-driven observability stack, there are a few problems that are necessary for a well-functioning environment [7]. If there are no metrics collected, the engineers will not be able to know how well the service is performing, or when there is degradation, before users are impacted. There are dashboards that get used during incident response, but can't be used unless they are visualised from the raw Prometheus queries. Fails could be transient and not be detected before the event unless they are reported

by the customer. If there is no infrastructure as code, the observability stack cannot be consistently replicated in environments which introduces operational risk.

Also, static AWS credentials, as repository secrets, can create a security risk in CI/CD pipelines. A pipeline used for building, pushing, and deployment of container images needs to authenticate itself towards AWS, and this should not happen using long-lived access keys, since it would be breaking the principle of least privilege and introduce an overhead of key rotation. This risk can be completely avoided by implementing a modern safe solution based on OpenID Connect and utilizing IAM role assumption [8].

1.3 Motivation

This work was motivated by a combination of a series of practical and academic factors. On a more practical level, Cloud-native observability is a skillset that is in demand. Engineers that can design and implement observability stacks top to bottom are valuable assets for any Ops or SRE team [9] and monitoring and alerting are consistent top-layer recurring feature areas of concern for organisations deployed on Kubernetes.

Academically, this work gives them an opportunity to practice the theory they have learnt in their distributed systems, networking, containerisation, and infrastructure automation classes on a continuous running system. It covers several domains such as cloud infrastructure provisioning, container orchestration, cloud instrumentation of applications, metrics collection, visualization, alerting and continuous delivery [10]. Creating a comprehensive and effective platform involves comprehending each of these domains and the interactions between them to ensure seamless integration.

The project was also driven by the need to operate in the realm of existing, industry standardized tools, and not with simplified versions found in academic settings. The technologies employed in this project are shipping currently being used by production engineering teams like Amazon EKS, Terraform, Helm, Prometheus, Grafana, and GitHub Actions..

1.4 Scope of the Project

This project is limited to the following scope. It installs two containerized applications into Amazon EKS – a Python FastAPI application instrumented with Prometheus metrics – and an NGINX-statically-hosted website. The observability stack includes Prometheus (metrics collection and storage), Grafana (visualisation and Golden Signals dashboards), Alertmanager (alert routing and SMTP notification), and Prometheus Blackbox Exporter (HTTP availability probing from the outside). All infra provisioning is done via terraform, and all application deployments are orchestrated via helm by a GitHub actions CI/CD pipeline.

Distributed tracing (OpenTelemetry, Jaeger, or Tempo), log aggregation (Loki, Elasticsearch, or Fluentd), or a service mesh (Istio, Linkerd) is not included in the project. These are recognized as being part of a broader observability strategy, and in no way a part of this particular implementation. The project is intended to limit scope to metrics pillar of observability — ideal to use external probe-based availability monitoring as a proxy for synthetic monitoring.

1.5 Applications

The platform and patterns demonstrated in this project have direct applicability in several contexts:

- Production Kubernetes deployments where engineering teams do not want to manage and maintain a proprietary monitoring platform, and want to have the essential features available.

- Golden Signals metrics (traffic, errors, latency, saturation) as the basis of service-level indicators and objects for Site Reliability Engineering (SRE) practices.

- DevOps teams that need to have a reproducible set of observability definitions from which they can easily reproduce.

- Organisations with legacy infrastructure monitoring solutions moving on-premises to cloud based solutions who need a reference implementation.

- Academic or training contexts where students need to learn how to observe a full working example of Kubernetes, not individual components.

Chapter 2: Literature Review

2.1 Existing Monitoring Systems and Approaches

There are several generations of monitoring in distributed systems [11]. The first generation tools, like Nagios and Zabbix, designed in the late '90s and 2000s, were based on agents or SNMP polling of hosts. These tools are aimed at rather stable infrastructure with limited endpoints that were known up front and rarely change. They were not configured by hand and did not natively support ephemeral or containerised workloads [12].

Second generation tools, such as Graphite, collectd and StatsD, were these, and they presented a messaging system for metrics (usually called a push-based system) where applications sent metrics to a central aggregator. This model was more flexible, but on the other hand, instruments had to be explicitly added to the application, and it had to be aware of the endpoint of the aggregator. To meet the needs of storage and querying high cardinality metrics at scale, new time-series databases like InfluxDB and TimescaleDB were introduced [13].

Some commercial monitoring-as-a-service platforms like Datadog, New Relic, Dynatrace, etc. provided wide coverage of instrumentation in containers, but created vendor dependency, licensing expenses, and data egress concerns that are unattainable in a wide variety of deployment scenarios. These platforms do simplify configuration but they do decrease the ability of the operator to control what is being monitored.

2.2 Cloud-Native Observability: Research and Practice

The idea of observability of software systems was popularised by Charity Majors and others who have been writing about the modern practices of operations. Observability introduces high dimensionality, high cardinality data capabilities that allow arbitrary runtime queries about system behaviour, rather than pre-defined dashboards and thresholds and alerting. The three pillars structure of data, namely metrics, logs and traces, are used to classify the data telemetry types.

Brendan Gregg's USE method (Utilisation, Saturation, Errors) and Google's Golden Signals framework (Traffic, Errors, Latency, Saturation) are both used as successful application metrics frameworks and first documented in the Google Site Reliability Engineering book. This project integrates Golden Signals into the Tiago Nemeth’s dial, which is done directly in the dashboard panels from the Grafana system.This project uses custom Grafana dashboard panels [14] for the integration of Golden Signals into the Tiago Nemeth's dial.

Studies related to Kubernetes-native monitoring have looked at how well the Prometheus scraping model performs with Kubernetes clusters, the cardinality problem caused by label explosion, and the reliability properties of the Alertmanager clustering model. Research studies generally have been able to show that the model of pull-based scraping promoted by Prometheus fits well into kubernetes environments, as it works with the kubernetes service discovery mechanisms, and has adequate overhead for medium sized clusters [15].

2.3 Infrastructure as Code and GitOps

Infrastructure as Code (IaC) is the ability to manage and provision computing infrastructure through machine-readable configuration files instead of through interactive configuration interfaces. HashiCorp Terraform is the world's most popular open-source IaC solution, features a provider-agnostic, declarative configuration language (HCL) and plans and applies changes incrementally in a state-based approach.

GitOps is an extension of the IaC principle, where the application and infrastructure state are viewed as both being the single source of truth in a Git repository [16]. Reconciliations on change are automated to get the state of the live system to match the state of the declared desired system. The researchers calculated 4 metrics that have always been strongly correlated to GitOps adoption by researchers and practitioners, and they come from such important metrics based on the DORA (DevOps Research and Assessment) research programme as deployment frequency, failure rate among changes, and mean time to recovery.

Helm is a package manager for Kubernetes that adds a layer of templating and lifecycle management to Kubernetes resource manifests, complementing Terraform [17]. The pattern of using Terraform for infrastructures and Helm for Kubernetes application deployment is used widely in production setup [18].

2.4 Limitations of Existing Solutions

Reviewing the existing landscape reveals several limitations that this project addresses:

- Most tutorials and reference implementation examples for observing Kubernetes deploy monitoring instrumentation manually using kubectl commands or Helm, without accounting for the infrastructure configuration in IaC. This results in a setup that cannot be reproduced and can be challenging to keep up.

- Current deployments tend not to implement the entire end-to-end infrastructure-to-deploy-to-observability pipeline in a coherent and unified codebase [19].

- Creating label matching for Prometheus ServiceMonitors, installation ordering of a Custom Resource Definition and Grafana sidecar loading of the dashboard is not well documented in detail and is likely to be solved through a bouncer of ideas on the producer end.

- Keyless CI/CD authentication using OpenID Connect is also often not made a part of the reference architecture in favor of simpler, though less secure, static credential approaches [20].

- An educational implementation that involves availability monitoring by an external component using the Blackbox Exporter is not well represented [21].

This project addresses each of these gaps through a complete, documented, and working implementation that integrates all components into a unified, code-driven system.

Chapter 3: Objectives and Methodology

3.1 Project Objectives

The objectives of this project are stated as follows:

1. Deploy a production-ready Amazon EKS cluster and all necessary networking resources (VPC, subnets, NAT Resource, Internet Gateway), IAM roles, ECR resources, and Bastion Host to enable cluster administration, all using Terraform.

2. Install and deploy 2 workloads, one as a microservice (FastAPI app with Prometheus metrics) and one as a static website (via NGINX), in the EKS cluster using Helm charts.

3. Deploy the full observability stack: the Prometheus (metrics with 7-day retention), Grafana (Golden Signals dashboard, pulling metrics from the sidecar), Alertmanager (SMTP email notification) and Prometheus Blackbox Exporter (provide completeness for probing from an external HTTP endpoint).

4. Determine and monitor custom PrometheusRule alerts for various metrics including high error rate, high P95 latency, service unavailability and external probe failure.

5. Using a complete GitHub Actions pipeline to automate the end-to-end build and deployment process using a keyless AWS authentication via OpenID Connect, eliminating all static credentials from the repository.

6. Use a single NGINX Ingress Controller LoadBalancer to expose all monitoring interfaces (Grafana, Prometheus, Alertmanager) as well as application endpoints (website, FastAPI) using path-based routing.

7. Document and record all engineering challenges, resolutions and learnings in a format accessible to those implementing similar systems.

3.2 System Methodology and Workflow

The project follows a layered provisioning methodology with two distinct Terraform execution layers and a GitHub Actions deployment pipeline. The workflow proceeds as follows:

3.2.1 Infrastructure Provisioning (Terraform infra layer)

The first execution layer, vida infra folder, creates all the AWS resources required. This layer is updated only at initial setup or when there are changes in the underlying infrastructure topology. Independent of application deployments means there are separate blast radii for different applications deployments and there are separate change management processes.

The infra layer deploys the following resources using modular Terraform configuration: a Virtual Private Cloud with the CIDR block of 10.0.0.0/16; two public and two private subnets spread across two AZs; an Internet Gateway and NAT Gateway to have access for egress; route tables and associations; an EKS cluster which has Kubernetes version 1.29 deployed; an EKS node group with m7i-flex.large snapshot instance, which is deployed on a scaling configuration of 1-3 nodes; IAM roles for EKS cluster and node group; IAM roles for GitHub actions OIDC;

Amazon ECRs for the FastAPI and website container images; and an EC2 instance running Bastion on a public subnet with kubectl and helm installed as user data..

3.2.2 Platform Component Deployment (Terraform apps layer)

The second execution layer sits underneath terraform/apps/, takes the outputs from the infra layer and deploys the platform layers in Kubernetes. This layer is responsible for the NGINX Ingress Controller, the kube-prometheus-stack Helm release (Prometheus, Grafana, Alertmanager, Node Exporter, kube-state-metrics), the Prometheus Blackbox Exporter, the monitoring namespace, Kubernetes custom resources (ServiceMonitor, PrometheusRule, Probe), Grafana dashboard ConfigMaps, and the aws-auth ConfigMap update for RBAC access.

3.2.3 CI/CD Pipeline (GitHub Actions)

A GitHub Actions workflow, defined in .github/workflows/deploy.yml should be responsible for the complete application build, and deployment. It runs when pushed to the main branch, doing the following steps in order: logging in to AWS using OIDC, logging into Amazon ECR, building and tagging Docker images with the Git Commit SHA, pushing images to ECR, updating kubeconfig, installing Helm, and applying Helm upgrades to both the FastAPI and website releases.

3.3 Block Diagram of the System

The overall system architecture can be understood as:

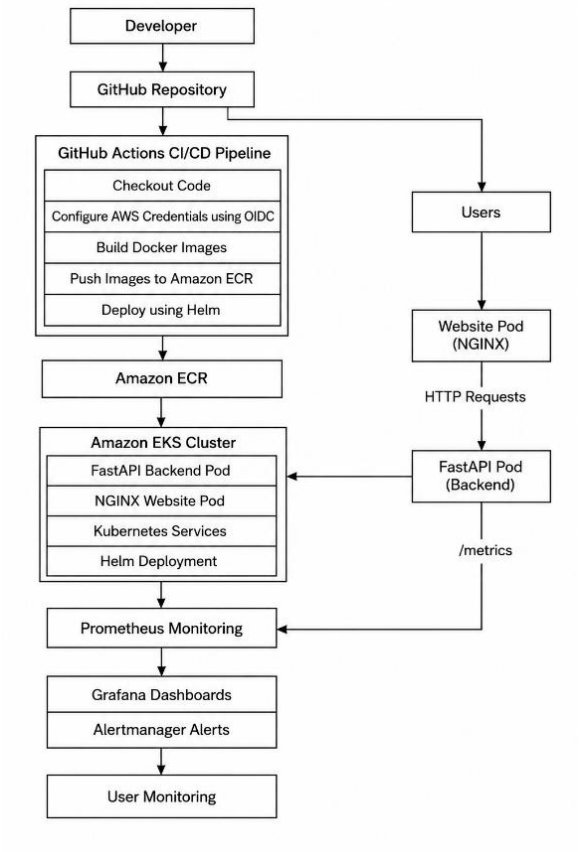

Figure 1: Block Diagram

3.4 Feasibility Study

3.4.1 Technical Feasibility

This project is using technologies that are production-proven, licensing options are open-source or free, and the technologies were tried and tested. All of the above are CNCF or HashiCorp projects with widespread documentation and documentation maintained and actively developed. Amazon EKS is a tool that takes care of the management of the control plane, in turn saving numerous operational challenges. GitHub Actions offers public repositories free minutes of CI/CD. The above project, therefore, is technically viable both for standard access to the University's computing services as well as using a personal AWS account.

3.4.2 Economic Feasibility

The main big ticket item is the Amazon EKS cluster and all of the Amazon EC2 node groups residing on this cluster. The managed control plane charges for an EKS cluster is USD 0.10 per hour. A total of two m7i-flex.large EC2 instances will cost about USD 0.19 per hour in use-east-1 region. A project can be developed and tested for several weeks with AWS, and overall AWS cost can be kept low within a reasonable student budget as the cluster can be destroyed between development sessions with the command terraform destroy.

3.4.3 Operational Feasibility

After provisioning, the system is to run without man intervention. The Terraform configuration will only deploy once and all future deployments of the application will include the CI/CD pipeline. Thus, the complexity of the operation is also limited to during the initial setup, and physics takes over within the Kubernetes control loop.

Chapter 4: System Design

4.1 Overall Architecture

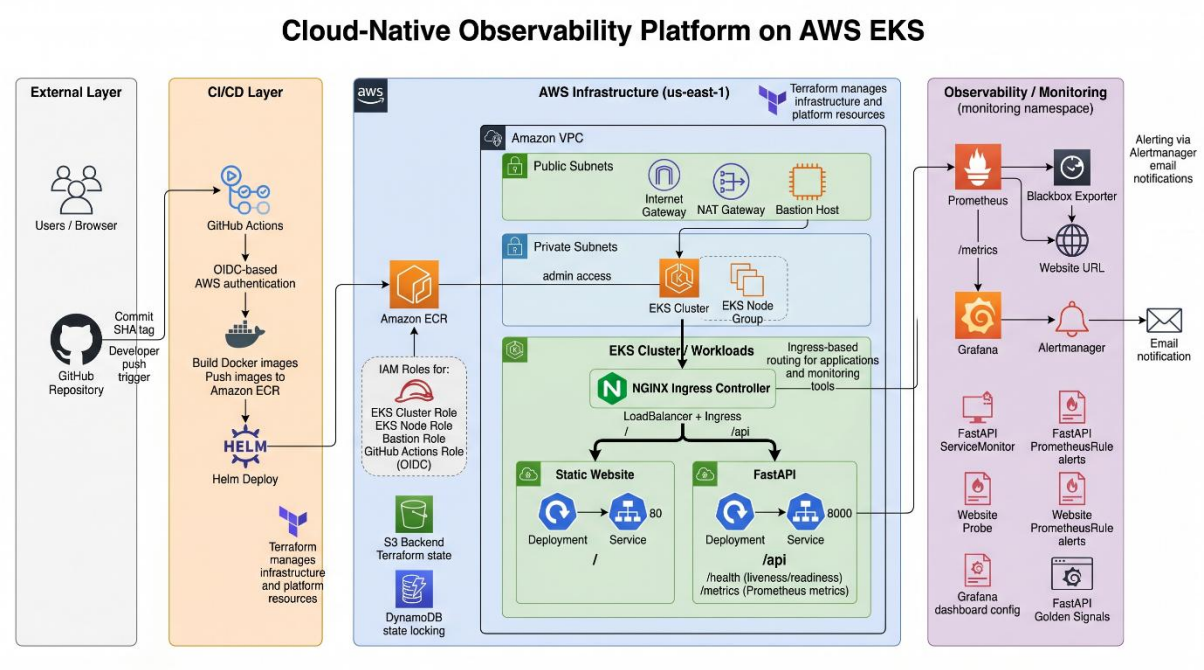

Figure 2: Architecture Diagram

The architecture of the platform consists of four different loosely coupled layers: The AWS infrastructure layer, the Kubernetes application layer, the observability layer, and the CI/CD automation layer. The interfaces between different layers are clearly defined, with one layer being independent of the other.

All of the Kubernetes resources are deployed on top of the AWS infrastructure layer, which is fully managed by Terraform. The Kubernetes application and observability layers are deployed within the EKS cluster and communicate with each other using the native Kubernetes mechanisms: Services for inter-pod communication; Ingress for traffic routing to the external; ServiceMonitor for registering the target for Prometheus scraping; ConfigMap for distributing dashboard and configurations. The CI/CD layer is outside of the cluster and communicates with the cluster through authenticated kubectl and Helm execution.

4.2 Networking Design

The VPC is configured with the CIDR block 10.0.0.0/16, providing 65,534 available IP addresses. The address space is divided into four subnets:

Subnet	CIDR	Availability Zone	Route Table
public-0	10.0.0.0/24	us-east-1a	Internet Gateway (0.0.0.0/0)
public-1	10.0.1.0/24	us-east-1b	Internet Gateway (0.0.0.0/0)
private-0	10.0.10.0/24	us-east-1a	NAT Gateway (0.0.0.0/0)
private-1	10.0.11.0/24	us-east-1b	NAT Gateway (0.0.0.0/0)

Pod network interfaces are not directly accessible from the internet by deploying EKS worker nodes in private subnets. The NAT Gateway is attached to the first public subnet, which is connected to the internet for outbound traffic for pulling container images from ECR and for Helm chart repository access from worker nodes. The Bastion Host and NGINX Ingress Controller LoadBalancer are deployed in the public subnets.

4.3 Kubernetes Cluster Design

The EKS cluster is eks-observability-cluster and is version 1.29 of Kubernetes. The control plane is fully managed by AWS and therefore won't be visible inside the cluster, which is relevant for the following expected Alertmanager alerts: KubeControllerManagerDown and KubeSchedulerDown. The data plane is an EKS managed node group of m7i-flex.large instance type and 2 nodes desired, 1 minimum, 3 maximum.

Workloads are spread across 2 namespaces: default for the application services (FastAPI, website); monitoring for all the observability components (Prometheus, Grafana, Alertmanager, Blackbox Exporter, Node Exporter, kube-state-metrics). This namespace separation offers logical isolation, which allows each namespace to have its own RBAC policies and resource quotas.

The NGINX Ingress Controller is deployed in the ingress-nginx namespace and is set to have a LoadBalancer service type resulting in an AWS Classic or Network Load Balancer configuration. This is the only load balancer that external traffic flows into the cluster; path-based load balancer rules direct the flow to the appropriate backend service.

4.4 Application Layer Design

4.4.1 FastAPI Backend

The FastAPI backend is a python3.11 application running on the Uvicorn ASGI server, on port8000. The application exposes 3 HTTP endpoints: GET / returns a basic health acknowledgement message, GET /health returns a Kubernetes status message (liveness and readiness probes), and GET /metrics returns Prometheus time-series metrics, generated by the prometheus-fastapi-instrumentator library.

To achieve basic HA with Helm chart, the FastAPI application is deployed with 2 replicas. The Kubernetes Service is of type ClusterIP and is listening on port 80, it targets the container port 8000. A named port http is a definition of a http port that is added to the Service to allow for ServiceMonitor target binding by the port name instead of port number, so that the scrape configuration will continue to work even if the port number changes.

Liveness and readiness probes set to using an HTTP GET /health on port 8000. The period of the readiness probe is 5 seconds and the initial delay is 10 seconds. The initial delay and period of liveness probe is 10 seconds and 20 seconds respectively. These values are configured to give the Uvicorn server enough time to start up before the probes start.

4.4.2 Static Website

Itis a single page HTML app running under NGINX Alpine. The Dockerfile pulls the index.html file to NGINX default HTML directory. The port 80 is open in the container. The Helm chart has 2 replicas and externalizes its service to ClusterIP with port 80. The site is a front facing application and is being used as the target for Blackbox Exporter availability probe..

4.5 Observability Layer Design

4.5.1 Prometheus

The prometheus monitoring is installed in the monitoring namespace as part of the kube-prometheus-stack Helm chart. The deployment config includes 1 replica, a 7 day retentionPeriod, and disables the flag serviceMonitorSelectorNilUsesHelmValues flag which causes Prometheus to query all ServiceMonitors across all namespaces, regardless of their Helm release label. This is required as the FastAPI ServiceMonitor is generated by the Terraform Apps Module and might not have the same Helm metadata as the kube-prometheus-stack Release.

The metrics are scraped from four types of targets: from the /metrics endpoint of the FastAPI server (collects basic metrics every 15 seconds via ServiceMonitor), from the metrics of the K8s nodes (via Node Exporter), from the state information of the K8s objects (via the kube-state-metrics binary), and from the Blackbox Exporter probe results (via the Probe custom resource). The Prometheus rules evaluation engine regularly checks Alert conditions defined in PrometheusRule resources.

4.5.2 Grafana

Grafana is set up to use the kube-prometheus-stack Prometheus instance as its primary datasource. The dashboard sidecar has been enabled and grafana_dashboard has been set to 1. The sidecar container discovers and loads automatically any ConfigMap stored in any namespace with this label, and containing a Grafana dashboard JSON file.

The custom FastAPI "Golden Signals" dashboard is saved as a JSON file in a Terraform managed "ConfigMap". The dashboard comprises of six panels: Request Rate (requests per second), Error Rate (5xx responses/second or error rates), P95 Latency (95th percentile response time), P99 Latency (99th percentile response time), FastAPI CPU / Disk Usage (container CPU/container memory disk core consumption), and FastAPI Memory Usage (Working Set in Mega Bytes). All panels are of the timeseries visualisation type and refresh every 10 seconds.

By setting root_url and serve_from_sub_path in grafana.ini, Grafana serves from a sub-path, such as /grafana, so not having a dedicated Grafana hostname is possible since it can be accessed via the shared NGINX Ingress Controller LoadBalancer.

4.5.3 Alertmanager

Prometheus sends the firing alerts to Alertmanager through the endpoint defined in the Prometheus configuration. The routing configuration assigns alerts to specific groups based on a matching alertname, defines an initial wait and aggregate interval time (10 seconds), defines a subsequent aggregate interval time (1 minute), and defines a repeat interval (1 hour) to prevent a number of subsequent alerts being sent when the alert has been turned on continuously.

There are two configurations of receiver - one to do nothing with alerts and another to send alerts as an e-mail using SMTP encryption via smtp.gmail.com on port 587. The alert is configured to send notifications to the email receiver which will also fire notifications, for when the alert resolves.

4.5.4 Blackbox Exporter

The Prometheus Blackbox Exporter is installed as a separate Helm release on the monitoring namespace. A Kubernetes Probe custom resource installs an HTTP probe that checks on the website service to see if it responds to a 30-second interval query. This target is being exported as the Blackbox Exporter exports it using the http_2xx module which returns anything with a 2xx HTTP response code as a successful response. The main Prometheus collects the probe results as metrics and stores them as Prometheus metrics.

4.6 IAM and Security Design

Identity and access management follows the principle of least privilege. Four IAM roles are provisioned:

IAM Role	Principal	Attached Policies	Purpose
eks-cluster-role	eks.amazonaws.com	AmazonEKSClusterPolicy	EKS control plane operations
eks-node-role	ec2.amazonaws.com	AmazonEKSWorkerNodePolicy, AmazonEKS_CNI_Policy, AmazonEC2ContainerRegistryReadOnly	Worker node operations and ECR image pulls
bastion-role	ec2.amazonaws.com	AmazonEKSClusterPolicy, eks:DescribeCluster inline	Administrative kubectl access from Bastion Host
github-actions-role	GitHub OIDC provider (sts.AssumeRoleWithWebIdentity)	AmazonEC2ContainerRegistryFullAccess, AmazonEKSClusterPolicy, AmazonEKSServicePolicy, eks:DescribeCluster inline	CI/CD pipeline: build, push images, deploy via Helm

The GitHub Actions role is assigned a trust relationship with the GitHub OIDC provider and is limited to the repository in question by using a condition on the sub claim of type StringLike. This means that we can only create workflows that stem from the githubabhay2003/BTech-Major-Project-Cloud-Native-Observability-EKS repository and not another repository within the same GitHub organisation so that there is no lateral movement.

4.7 CI/CD Pipeline Design

Each pipeline in GitHub Actions is written as a single pipeline like this .github/workflows/deploy.yml. It will only get triggered by push events to the main branch. The pipeline runs on ubuntu-latest runner and needs these two permissions: id-token: write for the OIDC token exchange and contents: read for repository checkout.

At a job level, the pipeline sets three kind of environment variables, AWS_REGION (us-east-1), ACCOUNT_ID (the ID of the AWS account build upon), and IMAGE_TAG, which is set to the GitHub SHA of the trigger commit. Because the image is always tagged with the commit SHA, every deployed version can be traced back to a particular commit to prove rollback and audit.

Here is the sequence for deploying: check out the code, configure AWS credentials using the OIDC, login to ECR, build the FastAPI image, tag & push the FastAPI image, build website image, tag & push the website image, update kubeconfig, install Helm, deploy FastAPI via Helm upgrade with dynamic repository and tag, deploy website via Helm upgrade with dynamic repository and tag.

Chapter 5: Implementation

5.1 Software Requirements

Tool / Service	Version / Specification	Role
Terraform	>= 1.4.0	Infrastructure as Code
AWS Provider (hashicorp/aws)	~> 5.0	Terraform AWS resource management
Amazon EKS	Kubernetes 1.29	Managed Kubernetes control plane
Amazon ECR	Managed service	Container image registry
Amazon S3	Managed service	Terraform remote state backend
Amazon DynamoDB	Managed service	Terraform state locking
EC2 (m7i-flex.large)	Amazon Linux 2023	EKS worker nodes
Docker	Latest	Container image build
Helm	3.x	Kubernetes package management
kubectl	Matching Kubernetes 1.29	Kubernetes cluster interaction
GitHub Actions	ubuntu-latest runner	CI/CD automation
Python	3.11-slim	FastAPI runtime
FastAPI	Latest stable	Backend API framework
Uvicorn	Latest stable	ASGI server
prometheus-fastapi-instrumentator	Latest stable	Prometheus metrics middleware
NGINX	Alpine variant	Static website web server
kube-prometheus-stack	81.2.2	Prometheus + Grafana + Alertmanager bundle
Prometheus Blackbox Exporter	Latest from prometheus-community chart	External HTTP probing
Git	Any recent version	Source control and CI trigger

5.2 Hardware Requirements

This project runs entirely on cloud-based infrastructure. The relevant hardware specifications are:

Resource	**Specification**
EKS Worker Node Instance Type	m7i-flex.large (2 vCPU, 8 GB RAM)
Node Count	2 (desired), with autoscaling from 1 to 3
EC2 Bastion Host	t3.micro (2 vCPU, 1 GB RAM), Amazon Linux 2023
Total Cluster vCPU	4 vCPU (2 nodes)
Total Cluster RAM	16 GB (2 nodes)
EBS Storage	Default gp3 EBS volumes per node (20 GB)
AWS Region	us-east-1 (N. Virginia)

5.3 Repository Structure

The project repository is organised as follows:

```
BTech-Major-Project-Cloud-Native-Observability-EKS/
├── .github/
│   └── workflows/
│       └── deploy.yml          # GitHub Actions CI/CD pipeline
├── apps/
│   ├── fastapi/
│   │   ├── Dockerfile
│   │   ├── main.py             # FastAPI application with /metrics
│   │   └── requirements.txt
│   └── website/
│       ├── Dockerfile
│       └── index.html
├── helm/
│   ├── fastapi/                # Helm chart: deployment, service, ingress
│   ├── website/                # Helm chart: deployment, service, ingress
│   ├── kube-prometheus-stack/
│   │   └── values.yaml
│   └── observability-ingress.yaml
├── terraform/
│   ├── infra/                  # Layer 1: VPC, EKS, IAM, ECR, Bastion
│   │   ├── modules/
│   │   │   ├── network/
│   │   │   ├── eks/
│   │   │   ├── iam/
│   │   │   ├── bastion/
│   │   │   └── ecr/
```

```
│   │   ├── github-oidc.tf
│   │   ├── main.tf
│   │   ├── outputs.tf
│   │   ├── variables.tf
│   │   ├── providers.tf
│   │   ├── backend.tf
│   │   └── versions.tf
│   └── apps/                  # Layer 2: Ingress, observability stack
│       ├── modules/
│       │   ├── ingress-nginx/
│       │   ├── observability/
│       │   └── observability-routing/
│       ├── kubernetes-auth.tf
│       ├── main.tf
│       ├── providers.tf
│       ├── remote-state.tf
│       └── backend.tf
└── docs/
    └── challenges-and-learnings.md
```

5.4 Infrastructure Implementation

5.4.1 Terraform Module: Network

Network module creates the VPC, subnets, Internet Gateway, NAT Gateway, Elastic IP and route table. The two public subnets and the two private subnets are created using Terraform count meta-arguments for resources. The availability zones are dynamically resolved with the help of aws_availability_zones data source to ensure the configuration changes when availability zones change for a particular region. Route table associations are the connection between the subnets and the route table (in this case, the connections are between a public subnet and the route table known as the Internet Gateway route and a private subnet and the route table known as the NAT Gateway route).

5.4.2 Terraform Module: EKS

The EKS module creates the aws_eks_cluster resource with the cluster role ARN and private subnet IDs, and the aws_eks_node_group resource with the node role ARN, subnet IDs, and scaling configuration. The node group uses the m7i-flex.large instance type, selected for its balance of compute and memory resources adequate to host the combined workload of application pods and monitoring components simultaneously.

5.4.3 Terraform Module: IAM

The IAM module creates the respective policy attachments for the cluster role and node role, and for the GitHub Actions role, creates an OIDC trust policy that is attached to the role. Minimal but sufficient access is granted to scope (sub) by the trust policy, which uses StringLike to bind it to the specific repository, and audience, which is bound to sts.amazonaws.com using the StringEquals policy.

5.4.4 Terraform Module: Bastion

The Bastion Host module installs the kubectl and Helm binary tools onto the newly-provisioned EC2 instance during its first boot. The instance is dynamically allocated the latest Amazon Linux 2023 AMI, has IAM instance profile attached with the bastion role and a security group is attached that allows SSH port 22 to be opened up from any IP address. A key pair is created automatically from a public key file located in the local (host) network. The Bastion Host is the recommended path to configure and administer aws-auth ConfigMap initially and in case of emergency.

5.4.5 Terraform Module: ECR

The ECR module sets up two repositories: eks-observability-fastapi and eks-observability-website. Both repositories use MUTABLE image tags that will be overwritten anytime the image is modified during development and within the repositories have the scan-on-push Boolean set to true, which will enable basic vulnerability scanning. The flag force_delete is set to true to enable Terraform to delete repositories even when there are images in them (which may be used for testing purposes.

5.5 Application Implementation

Figure 3: Grafana dashboard visualizing FastAPI Golden Signals — request rate, latency (P95/P99), error rate, and resource usage (CPU & memory) in real time

5.5.1 FastAPI Application (main.py)

The FastAPI application is minimal by design, serving as a representative instrumented microservice rather than a production business application. The critical implementation detail is the Instrumentator initialisation:

```
from fastapi import FastAPI
from prometheus_fastapi_instrumentator import Instrumentator
```

```
app = FastAPI()

@app.get("/")
def root():
    return {"message": "FastAPI Observability App"}

@app.get("/health")
def health():
    return {"status": "ok"}

Instrumentator().instrument(app).expose(app)
```

The Instrumentator().instrument(app).expose(app) call adds three metrics to the application: Counter: http_requests_total:method/handler:status: Metric that counts the number of HTTP requests; Histogram: http_request_duration_seconds:method/handler: Metric that measures the time spent on each HTTP request as the duration in seconds; Gauge: http_requests_in_progress:method/handler: Metric that shows how many HTTP requests are currently in progress. These metrics are exposed on the /metrics endpoint in Prometheus exposition format.

5.5.2 FastAPI Dockerfile

The FastAPI Docker image is based on: python:3.11-slim, this reduces the image size to avoid development dependencies. The Dockerfile copies the requirements.txt first, and then installs dependency from that file, by leveraging the "layer-by-layer" caching provided by Docker, dependency installation is only re-executed if requirements.txt changed. The container is set up to use Uvicorn with 0.0.0.0:8000.

5.5.3 FastAPI Helm Chart

The FastAPI Helm chart exposes three Kubernetes resource templates: Deployment, Service, Ingress. The replica count, image repository, image tag, image pull policy, service target port and app label params are taken from values.yaml for the Deployment template. This parameterisation is important for the CI/CD pipeline: when you deploy, the CI/CD pipeline, with --set flags, will override - image.repository and image.tag.

The Service template defines a ClusterIP service with a port named "http", with port 80 assigned to the container port 8000. ServiceMonitor configuration uses the port name and not the port number. Ingress side rules setup in Ingress template will include nginx ingressClassName with a rewriteTarget: regex path in the annotation to enable rewrite rule that will rewrite all requests to /api and /api/* to / and /* respectively respectively before reaching the FastAPI service.

5.6 Observability Implementation

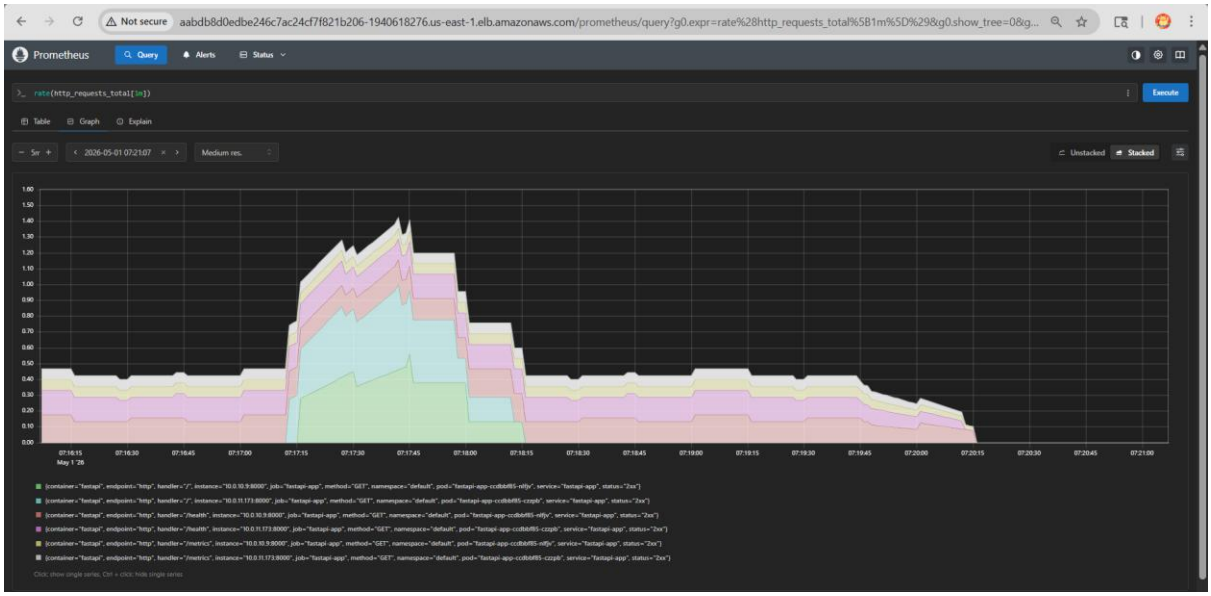

Figure 4: Prometheus querying real-time metrics (request rate) from instrumented FastAPI service

5.6.1 kube-prometheus-stack Helm Values

The kube-prometheus-stack Helm release is configured with a detailed values override in the Terraform Helm resource. Key configuration decisions include:

- Grafana admin credentials set to admin/prom-operator for initial access.
- Grafana serve_from_sub_path enabled with root_url pointing to the LoadBalancer hostname and /grafana path, enabling sub-path access without redirect failures.
- Grafana sidecar configured with label grafana_dashboard: 1 and searchNamespace: ALL, enabling automatic discovery of dashboard ConfigMaps across all namespaces.
- Prometheus serviceMonitorNamespaceSelector set to an empty object, overriding the default Helm-scoped selector and allowing Prometheus to discover ServiceMonitors cluster-wide.
- Alertmanager SMTP configuration with Gmail SMTP relay, including application-specific password for OAuth2-secured Gmail accounts.
- prometheusBlackboxExporter enabled as part of the kube-prometheus-stack release, providing the HTTP probe module.

5.6.2 ServiceMonitor

ServiceMonitor is a custom resource which is created as a Kubernetes manifest resource in Terraform. It looks up the FastAPI service named "app: fastapi" in the default namespace, creates a scrape endpoint for the http named port at the /metrics path with a 5-minute frequency and assigns the release: kube-prometheus-stack label that will be matched by the Prometheus instance's ServiceMonitor selector.

5.6.3 PrometheusRule Definitions

Three alert rules are defined for the FastAPI service and one for the website:

Alert Name	PromQL Expression	For Duration	Severity
FastAPIHighErrorRate	sum(rate(http_requests_total{status=~"5.."}[1m])) > 1	1 minute	warning
FastAPIHighLatency	histogram_quantile(0.95, sum(rate(http_request_duration_seconds_bucket[5m])) by (le)) > 1	1 minute	warning
FastAPIDown	kube_deployment_status_replicas_available{namespace="default", deployment="fastapi-app"} == 0	30 seconds	critical
WebsiteDown	probe_success{instance="http://website.default.svc.cluster.local:80/"} == 0	30 seconds	critical

5.6.4 Grafana Dashboard ConfigMap

The Golden Signals dashboard JSON is mounted as a kuberneretes_config_map_v1 resource with the label grafana_dashboard: 1 attached to the Terraform resource. The queries used in the JSON file represent six timeseries panels for the application metrics collected as the FastAPI application. It is injected into the monitoring namespace and the Grafana sidecar container copies the JSON file to the Grafana dashboards directory and registers the file with the Grafana dashboard provider when the cluster starts up.

5.6.5 Ingress Routing for Monitoring Tools

There are two ways for monitoring tools in the monitoring namespace to be exposed to be made available. The observability-routing Terraform module will create ExternalName services in the default namespace that proxy into monitoring namespace services and then create Ingress resources that route /grafana and /prometheus paths to the proxy services. The /alertmanager path is handled by a separate Ingress with a rewrite annotation; the path needs to be path stripped first before being forwarded to the Alertmanager API.

5.7 CI/CD Pipeline Implementation

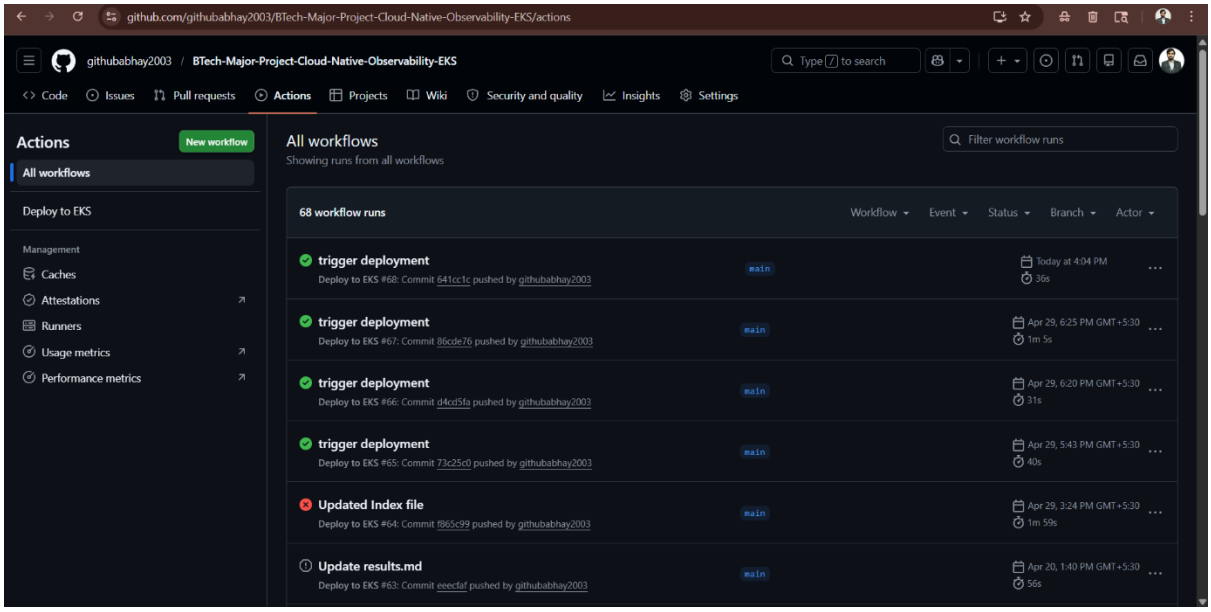

Figure 5: GitHub Actions pipeline automatically building, pushing to ECR, and deploying to EKS

The deploy.yml GitHub Action is a complete example of deployment automation. They all form part of the critical sequence that is required to ensure secure authentication to AWS services: aws-actions/configure-aws-credentials@v4 - role-to-assume is the parameter being passed here for the AWS IAM role ARN. Nothing is stored in the repository that contains AWS_ACCESS_KEY_ID or AWS_SECRET_ACCESS_KEY secrets. The OIDC token exchange is managed by the GitHub actions run infrastructure and is not visible from the outside.

Using image tagging to generate a Git commit SHA (${{ github.sha }}) will make sure that each build of your image will be uniquely tagged. An image tag change is something that passes through commit SHA through the Helm upgrade command's --set image.tag.Commit SHA is an image tag change that passes through the Helm upgrade command --set image.tag value which Kubernetes uses to detect the image change in order to roll the image updates in the targeted pods. The --install mode on helm upgrade guarantees that the first time it's run, it will make the Helm release; the second time run, it will upgrade the Helm release.

Chapter 6: Results and Discussion

6.1 Infrastructure Provisioning Results

After successfully provisioning all AWS resources in around 12-18 minutes, Terraform apply on the infra layer completes the provisioning of resources.All AWS resources are provisioned after the infra layer completes the provisioning in around 12-18 min, with the EKS cluster creation taking the longest to provision and the provisioning of the NAT Gateway taking another minute to complete. We have a total of 47 resources in the 5 modules that are listed in the Terraform state file on S3. The outputs published by this layer (vpc_id, public_subnet_ids, private_subnet_ids, eks_cluster_name, bastion_public_ip, ecr_repository_url, eks_node_role_arn, bastion_role_arn) are consumed by the apps layer via terraform_remote_state.

Post-provisioning verification shows that the two EKS worker nodes are in Ready status roughly 3 minutes after the node group is in ACTIVE status. The kubectl get nodes command run from the Bastion Host verifies that nodes are ready:

```
NAME                          STATUS   ROLES    AGE   VERSION
ip-10-0-10-xxx.ec2.internal   Ready    <none>   3m    v1.29.x-eks-xxx
ip-10-0-11-xxx.ec2.internal   Ready    <none>   3m    v1.29.x-eks-xxx
```

6.2 Observability Stack Results

The kube-prometheus-stack Helm release installs fine in monitoring namespace. The following pods go into Running status in 3–5 minutes:

Pod Name Pattern	Containers	Status	Port
prometheus-kube-prometheus-prometheus-0	2/2 (prometheus + config-reloader)	Running	9090
alertmanager-kube-prometheus-alertmanager-0	2/2 (alertmanager + config-reloader)	Running	9093
kube-prometheus-grafana-*	3/3 (grafana + sidecar + init)	Running	3000
prometheus-node-exporter-*	1/1	Running (DaemonSet)	9100
kube-state-metrics-*	1/1	Running	8080
prometheus-blackbox-exporter-*	1/1	Running	9115

By navigating to the Grafana UI at the LoadBalancer endpoint with /grafana on the end, you realize that the sidecar has automatically loaded the Golden Signals dashboard created with FastAPI. The data are shown for all six panels after the first scrape interval (15 seconds) after FastAPI was deployed.

6.3 Application Deployment Results

The awesome GitHub Actions pipeline runs successfully the first time we push the code to our main branch. The images for FastAPI and website are built and incorporated with the proposed SHA of the commit and pushed to their respective ECR repositories in the combined time of about 90 seconds.About 90 seconds is taken to build the images for FastAPI and website, tag them with the suggested commit SHA and push them to their respective ECR repositories. Both will be deployed to the default namespace by Helm. Mallows end up at Running status a few pods later:

```
NAME                          READY   STATUS    RESTARTS   AGE
fastapi-app-6d7c9b8f4-p2qrs   1/1     Running   0          2m
fastapi-app-6d7c9b8f4-wkt4x   1/1     Running   0          2m
website-5f8d9c7b3-mnxvk       1/1     Running   0          2m
```

The NGINX Ingress Controller LoadBalancer endpoint is pointed at the website on path /, the API behind FastAPI on path /api, Grafana on /grafana, Prometheus on /prometheus and Alertmanager on /alertmanager. Each and every five routes are verified as returning expected HTTP responses.

6.4 Metrics Collection Results

The Prometheus targets page at /prometheus/targets confirms that the FastAPI ServiceMonitor target is in the UP state and being scraped every 15 seconds. The following metrics are confirmed present in Prometheus after initial scraping:

- http_requests_total labeled/bounded by the HTTP method, handler, and status.
- Labels for http_request_duration_seconds using a label for method and handler bucket and count respectively.
- Labels for http_requests_in_progress with method and handler.
- The following stats from the Blackbox Exporter for the website target: probe_success and probe_duration_seconds.
- The Level 2 metrics include node_cpu_seconds_total, node_memory_MemAvailable_bytes, and others from the Node Exporter program.
- It currently includes kube_deployment_status_replicas_available, kube_pod_info and other kube-state-metrics.

6.5 Alerting Results

6.5.1 Expected Alerts in Amazon EKS

The following alerts fire immediately upon deployment and are expected behaviour in a managed EKS environment:

- KubeControllerManagerDown: This is because the Kubernetes controller manager is not exposed to the cluster as a scrape target, it is run within the AWS-managed control plane. This alert is triggered on an ongoing basis, and is a common false positive with managed EKS deployments.

- KubeSchedulerDown: The same reasoning applies to the Kubernetes scheduler.

- Watchdog: A rule in the default kube-prometheus-stack that will always trigger an alert to check that the pipeline from Prometheus to alerting is up and running end to end.

These alerts will go to the set e-mail recipient and will be recorded as informational in the project challenge and learning file. They do not indicate system malfunction.

6.5.2 Custom Alert Validation

Custom alert rules are validated through deliberate failure injection:

- According to the FastAPIDown alert, replicas of the deployment are changed to 0 replicas when scaling the FastAPI deployment to 0 replicas (kubectl scale deployment fastapi-app --replicas=0), which leads to the kube_deployment_status_replicas_available metric to reach 0. The alert runs once the set time for duration, that is, 30 minutes, has elapsed and an email alert was received to the set address within about 90 minutes after the scale-down operation. After scaling replicas up (using "kubectl scale deployment fastapi-app --replicas=2"), you'll receive a resolved notification when pods go into Running state again.

- If the deployment of the website is scaled down to 0 replicas, it seems the Blackbox Exporter probe fails (probe_success == 0). Alert is triggered after 30 seconds, and an email message is sent. When restoration is done a resolved notification is created.

- These alerts are verified using a load generator tool by pushing synthetic load and failure conditions into the system: FastAPIHighLatency and FastAPIHighErrorRate. The duration alert (and duration notification) are triggered for each of the duration alerts set and send e-mail notification.

6.6 Performance Evaluation

The cluster gets a bit of overhead from the observability stack, but not too much. Under normal usage, Prometheus with the configured scrape targets and 7-day retention requires some 1-2 GB of memory. Grafana will consume around 200MB. On 2-nodes configuration combined monitoring namespace pod set is around 3 to 4 vCPU seconds per minute and 3 to 4 GB of memory in steady state which equals to around 30 to 40 percent of total cluster capacity.

As a cluster gets larger, the monitoring would be a smaller percentage of the resources allocated to the cluster. It is designed to be very smart of resources, with single replicas for Prometheus, Alertmanager in the academic environment. This, when deployed for a production, would scale

the number of Prometheus replicas and add a persistent volume claim to store metrics data with a retention beyond the 7 days of in-memory storage.

6.7 Discussion

The outcomes are supporting the project objective. The infrastructure provisioning is accomplished using Terraform configuration and includes no manual steps other than an initial setup of the AWS account and generation of an SSH key. The CI/CD pipeline makes reliable end-to-end deployments and all observability components are working as expected.

One important discovery in this project is how difficult it is to integrate Prometheus with the ServiceMonitor/PrometheusRule custom resources. More specifically, there are ways that serviceMonitorSelectorNilUsesHelmValues, the ServiceMonitor release label and the Prometheus selector configuration must align for the scrape target to be automatically created. Details appropriately documented in project's challenges log.

One additional security feature highlighted is the use of the CI/CD authentication, which is based on OpenID Connect, and would not typically be demonstrated when using static credentials. Once created the OIDC configuration relies on parameters and operators that require some degree of precision of the IAM identity provider (the audience and the subject are handled by StringEquals and StringLike, respectively) and upon once properly constructed it works invisibly and avoids credential rotation time and resources altogether.

Chapter 7: Key Challenges and Resolutions

The implementation of this project encountered numerous engineering challenges across infrastructure provisioning, Kubernetes configuration, CI/CD authentication, and observability stack integration. This chapter documents the most significant challenges and their resolutions. A comprehensive catalogue of over one hundred challenges is available in docs/challenges-and-learnings.md in the project repository.

#	Challenge	Root Cause	Resolution
01	Cross-Module Terraform Ressource Referencing	Terraform modules cannot directly reference resources in sibling modules	Expose required values as module outputs and pass them as explicit input variables
02	EKS RBAC Access Failure for IAM Users	IAM roles not mapped in aws-auth ConfigMap	Update aws-auth ConfigMap with correct mapRoles entries for node role, bastion role, and GitHub Actions role
03	Remote State Dependency Misconfiguration	Incorrect backend config in terraform_remote_state block	Align bucket, key, and region values between infra backend.tf and apps remote-state.tf
04	CRD Not Installed Before Custom Resource Creation	ServiceMonitor and PrometheusRule applied before CRD installation by kube-prometheus-stack	Add depends_on referencing the kube-prometheus-stack Helm release to all Kubernetes manifest resources
05	FastAPI Metrics Endpoint Not Exposed	No /metrics endpoint in the default FastAPI application	Add prometheus-fastapi-instrumentator to requirements.txt and call Instrumentator().instrument(app).expose(app)
06	Kubernetes Label Case Sensitivity	app: FastAPI in values.yaml did not match app: fastapi in ServiceMonitor selector	Standardise all labels to lowercase across Deployment, Service, and ServiceMonitor resources
07	ImagePullBackOff on Worker Nodes	ECR URI format incorrect or AmazonEC2ContainerRegistryReadOnly policy not attached to node role	Verify ECR URI format (account.dkr.ecr.region.amazonaws.com/repo:tag) and confirm policy attachment in IAM module

08	Static AWS Credentials in GitHub Actions	Initial pipeline used AWS_ACCESS_KEY_ID and AWS_SECRET_ACCESS_KEY repository secrets	Implement OIDC-based authentication using aws-actions/configure-aws-credentials@v4 with role-to-assume
09	Helm Release Stuck in Failed State	Partial Helm deployment left stale release metadata	Manually delete failed release with helm uninstall and re-run pipeline
10	terraform destroy Blocked by AWS Dependencies	ELB and ENI resources created by Kubernetes remained attached	Delete Kubernetes Ingress and LoadBalancer services before running terraform destroy
11	Prometheus Not Scraping FastAPI Metrics	serviceMonitorSelectorNilUsesHelmValues defaulted to true, limiting ServiceMonitor selection to Helm-scoped resources	Set serviceMonitorNamespaceSelector to empty object and serviceMonitorSelectorNilUsesHelmValues to false in Prometheus spec
12	Grafana Dashboard Not Auto-Loading	ConfigMap label used grafana_dashboard key with underscore, but Grafana sidecar expected camelCase	Align label key in ConfigMap with the label value configured in the Grafana sidecar.dashboards.label field
13	Alertmanager SMTP Authentication Failure	Regular Gmail password rejected; Gmail requires application-specific passwords for SMTP	Generate a Gmail application-specific password and use it in smtp_auth_password configuration
14	Grafana Sub-Path Access Returning 404	Grafana redirected to root URL without sub-path prefix, causing 404 on NGINX Ingress	Configure grafana.ini server section with root_url and serve_from_sub_path in Helm values
15	Blackbox Probe Target Not Discovered	Probe custom resource did not carry the release: kube-prometheus-stack label required by the Prometheus probeSelector	Add release: kube-prometheus-stack label to the Probe resource metadata

Chapter 8: Conclusion and Future Scope

8.1 Conclusion

In this project, we successfully designed, implemented and validated a production-grade Cloud-Native Observability Platform that is deployed on Amazon Elastic Kubernetes Service. It meets the stated goals: The infrastructure itself is provisioned through Terraform, and there is no manual interaction with the cloud console, the application deployments are fully automated, fed with an AWS OpenID Connect-based authentication, a complete observability stack is defined as Prometheus, Grafana, Alertmanager and Blackbox Exporter was up and running, and alert rules defining four main failure modes for this web service are configured and tested to generate SMTP email notifications in case of failure.

The project illustrates how—and why—it is possible, using Terraform (infrastructure-as-code), Helm (kubernetes package manager), and the kube-prometheus-stack (observability), to create a coherent, maintainable, and reproducible platform that follows the best practices of modern SREs. The modular Terraform structure comprising different infra and apps execution layers allows for independent lifecycle management of infra- and app-composition to diminish the risk of unwanted side-effects of either change class.

A major consequence of this project is the thorough and meticulous recording of the problems that were faced in trying to integrate them together. However, there exists several points of friction between Prometheus service discovery, Kubernetes custom resource selectors and Helm release label conventions which aren't fully documented. The challenges-and-learnings repository document requires fixing; it has a problem description set for the challenges, and a remedies set for the learnt solutions (which can be found to be correct or not), that is concrete.

That capability has centralized authentication on CI/CD platforms, such as AWS, with OpenID Connect instead of the more commonly proven static credential approach – an important security enhancement, which is becoming a standardization of production environments, and which is actually quite easy to deploy via the GitHub Actions OIDC provider and the aws-actions/configure-aws-credentials action.

8.2 Achievements

- Fully operational EKS code provisioned cluster featuring a VPC, two public subnets, two private subnets, an Internet Gateway, NAT Gateway, and Bastion Host.
- The microservices (FastAPI and NGINX website) are deployed as containerised services using Helm and get automatic image updates on every git push.
- Application metrics being scraped from Prometheus every 15 seconds and stored for 7 days.
- Real-time visualisation of request rate, error rate, P95 and P99 latencies, CPU usage and memory usage was created in a Golden Signals Grafana dashboard, which has been automatically deployed from a version-controlled ConfigMap.

- Alertmanager setup with SMTP email routing, with four custom alert conditions triggering firing and resolved emails.
- HTTP availability probing of the website service is being performed by the Blackbox Exporter with automatic alert on failure.
- Zero stored credentials in the repository with a GitHub Actions CI/CD pipeline implemented with OIDC authentication that does not store any credentials inbetween.
- Proving solutions to fifteen different engineering challenges and storing a large number of more minor challenges in the repository.

8.3 Limitations

The following limitations are acknowledged:

- Observability implementation only includes the metrics pillar. No log aggregation (Loki, Fluentd, or Elasticsearch) or distributed tracing (OpenTelemetry with Jaeger or Tempo) support, thus limiting the root cause analysis of complex failures that involve multiple services to the level of metrics.
- Prometheus storage – ephemeral storage with a retention of 7 days, in memory. Metric history is lost if the pod restarts or fails for any reason. To provide a production deployment, persistent volumes has to be backed by EBS or add a long-term storage backend like Thanos or Cortex with the remote write configuration.
- The Gmail application password is saved in the Alertmanager SMTP configuration file. For production, the credential should be stored directly inside AWS Secrets Manager, and use the ExternalSecret operator or similar to inject it.
- The FastAPI application does not provide any authentication layer. In an Ingress, all API endpoints are open for business. Authenticating in the production level, at the Application or Ingress level is required.
- A single NAT Gateway serves the entire cluster, and provides the sole path for outbound Internet traffic from private subnets, creating a single point of failure for outbound Internet connection. Given a Production Deployment, it will provision a NAT Gateway in each AZ.
- This project did not include load-testing and sustained performance measurements under realistic traffic volumes.

8.4 Future Enhancements

The following enhancements are proposed for future development:

- Installing Grafana Loki stack with Prometheus serving as a log collection agent is being implemented via Log Aggregation. It would finish the metrics and logs pieces of the observability approach and allow for logs and metrics to be linked within Grafana.

- Distributed Tracing: Use OpenTelemetry to instrument the FastAPI application and deploy OpenTelemetry as a trace backend, Tempo, or Jaeger. It is possible to connect traces directly from Grafana metrics panels thanks to the unified datasource model of Grafana.

- Persistent Metrics Storage: Add support for prometheus remote-write to Thanos or Cortex cluster backed by S3, allowing to extend the metrics' life-cycle beyond 7 days and provide high availability query federation.

- Horizontal Pod Autoscaling: Configure HPO for the deployment of FastAPI with Custom Metrics (e.g. request rate) in Kubernetes exposed by Prometheus Adapter, for traffic driven autoscaling.

- Service Mesh Integration: Implement mutual TLS, advance traffic management (canary releases, circuit breaking) and generate L7 metrics without any instrumentation changes in the application.

- Multi-Environment Terraform Workspaces: Support multiple environments (dev, staging, production) with Terraform configuration via Terraform workspaces or separate Terraform state backend keys to test parity between environments.

- Support GitOps with ArgoCD, declarative GitOps reconciliation and drift detection of Kubernetes manifests instead of the Helm based one..

- Manage AWS Secrets Manager with the External Secrets Operator for Alertmanager SMTP credentials, Grafana admin passwords and other critical configuration values that aren't set in Terraform state.

References

[1] Y. M. Abgaz *et al.*, "Decomposition of Monolith Applications Into Microservices Architectures: A Systematic Review," *IEEE Transactions on Software Engineering*, pp. 1–32, Jan. 2023, doi: https://doi.org/10.1109/tse.2023.3287297.

[2] E. Paintsil, "Introduction to the Amazon Elastic Kubernetes Service," *AWS EKS Essentials*, pp. 3–11, 2025, doi: https://doi.org/10.1007/979-8-8688-1331-3_1.

[3] Dr. Mehdi Ghane, "Observability Engineering Fundamentals," *Apress eBooks*, pp. 83–147, Jan. 2025, doi: https://doi.org/10.1007/979-8-8688-1258-3_4.

[4] Kowshik Sakinala, "Monitoring and Observability for Cloud-Native Applications," *Journal of Computer Science and Technology Studies*, vol. 7, no. 8, pp. 101–115, 2025, doi: https://doi.org/10.32996/jcsts.2025.7.8.13.

[5] M. Chakraborty and A. P. Kundan, *Monitoring Cloud-Native Applications*. Berkeley, CA: Apress, 2021. doi: https://doi.org/10.1007/978-1-4842-6888-9.

[6] "Cloud-based Observability in Distributed Systems: Designing a scalable observability architecture in the cloud using Azure, OpenTelemetry and. NET," *Diva-portal.org*, 2026. https://www.diva-portal.org/smash/record.jsf?pid=diva2:1964791 (accessed May 22, 2026).

[7] "Google Books," *Google.com*, 2019. https://books.google.com/books?hl=en&lr=&id=Lt9EEAAAQBAJ&oi=fnd&pg=PP16&dq=The+absence+of+a+structured (accessed May 22, 2026).

[8] Surendra Vitla, "Securing the physical and digital frontier: leveraging identity and access management (IAM) to address the lack of controls on physical access to sensitive systems," *International Journal of Science and Research Archive*, vol. 6, no. 2, pp. 108–125, Aug. 2022, doi: https://doi.org/10.30574/ijsra.2022.6.2.0142.

[9] "A Survey on Observability of Distributed Edge & Container-Based Microservices," *ieeexplore.ieee.org*. https://ieeexplore.ieee.org/abstract/document/9837035/

[10] P. Raj and A. Raman, "Automated Multi-cloud Operations and Container Orchestration," *Software-Defined Cloud Centers*, pp. 185–218, 2018, doi: https://doi.org/10.1007/978-3-319-78637-7_9.

[11] J. Joyce, G. Lomow, K. Slind, and B. Unger, "Monitoring distributed systems," *ACM Transactions on Computer Systems*, vol. 5, no. 2, pp. 121–150, Mar. 1987, doi: https://doi.org/10.1145/13677.22723.

[12] R. Vaño, I. Lacalle, Piotr Sowiński, Raúl S-Julián, and C. E. Palau, "Cloud-Native Workload Orchestration at the Edge: A Deployment Review and Future Directions," *Sensors*, vol. 23, no. 4, pp. 2215–2215, Feb. 2023, doi: https://doi.org/10.3390/s23042215.

[13] E. Duarte, D. Gomes, D. Campos, and R. L. Aguiar, "Scalable Architecture, Storage and Visualization Approaches for Time Series Analysis Systems," *Communications in computer and information science*, pp. 59–82, Jan. 2020, doi: https://doi.org/10.1007/978-3-030-54595-6_4.

[14] “Monitoring Integration Systems and Visualization.” Available: https://www.utupub.fi/bitstream/handle/10024/154329/Vuorinen_Simo_opinnayte.pdf?sequence=1

[15] “Mastering Prometheus,” *Google Books*, 2024. https://books.google.com/books?hl=en&lr=&id=vhr9EAAAQBAJ&oi=fnd&pg=PP1&dq=Studies+have+generally+confirmed+that+the+pull-based+scraping+model+introduced+by+Prometheus+is+well-suited+to+Kubernetes+environments+due+to+its+alignment+with+Kubernetes+service+discovery+mechanisms (accessed May 22, 2026).

[16] Surbhi Kanthed, “From Code to Cloud: The Role of GitOps, GitHub, and GitLab in Modern DevOps,” *Journal of Technological Innovations*, vol. 6, no. 1, 2025, doi: https://doi.org/10.93153/851an548.

[17] M. Schumann, “Conceptual design of a container-based system landscape orchestrated by Kubernetes,” *Libdoc.whz.de*, 2024, doi: https://libdoc.whz.de/opus4/files/17421/BachelorThesisMaximilianSchumann.pdf.

[18] “Kubernetes in Production Best Practices,” *Google Books*, 2021. https://books.google.com/books?hl=en&lr=&id=y3MeEAAAQBAJ&oi=fnd&pg=PP1&dq=The+combination+of+Terraform+for+infrastructure+provisioning+and+Helm+for+Kubernetes+application+management+is+a+widely+adopted+pattern+in+production+environments.&ots=lWfEZhpsKG&sig=T1Jpd8alm-3L_x0BDV3oGPZSc7Q (accessed May 22, 2026).

[19] Y. Yuan, “From Data Pipelines to Platform Tooling: A Holistic Framework for Cloud-Native Backend Development,” *Journal of Computer, Signal, and System Research*, vol. 3, no. 1, pp. 101–114, 2026, doi: https://doi.org/10.71222/60vfc629.

[20] S. M. Saleh, N. H. Madhavji, and J. Steinbacher, “Systematic Review of Identity-Centric Security in Cloud-Native CI/CD Pipelines,” *Proceedings of the 2025 10th International Conference on Cloud Computing and Internet of Things*, pp. 23–32, Nov. 2025, doi: https://doi.org/10.1145/3785520.3785525.

[21] M. A. Anjum, “Platform engineering and internal developer portals: a multivocal literature review,” *Frontiers in Computer Science*, vol. 8, May 2026, doi: https://doi.org/10.3389/fcomp.2026.1814498.

[22] “Monitoring with Prometheus,” *Google Books*, 2018. https://books.google.com/books?hl=en&lr=&id=EtlfDwAAQBAJ&oi=fnd&pg=PA1&dq=prometheus+documentation&ots=57fGFF5m3f&sig=KpNeLBLgQENj49ojk_8p8z-zmWo (accessed May 22, 2026).

[23] “Learning Amazon Web Services (AWS),” *Google Books*, 2019. https://books.google.com/books?hl=en&lr=&id=HvifDwAAQBAJ&oi=fnd&pg=PT25&dq=aws+documentation&ots=-2jkRUsfL0&sig=dXEHKgpSOMKB_VGaKJb--IBiFj8 (accessed May 22, 2026).

[24] M. Holopainen, “Monitoring Container Environment with Prometheus and Grafana,” *Theseus.fi*, 2021, doi: https://www.theseus.fi/handle/10024/497467.

[25] T. Leppänen, “Data visualization and monitoring with Grafana and Prometheus,” *www.theseus.fi*, 2021. https://www.theseus.fi/handle/10024/512860

[26] “Learning Helm,” *Google Books*, 2021. https://books.google.com/books?hl=en&lr=&id=ljYWEAAAQBAJ&oi=fnd&pg=PT12&dq=helm+documentation&ots=xRWdZgFXjX&sig=jBEapZgeCSsHFWp6IMs3jVlpAa4 (accessed May 22, 2026).

[27] “Terraform: Up and Running,” *Google Books*, 2022. https://books.google.com/books?hl=en&lr=&id=dFaKEAAAQBAJ&oi=fnd&pg=PT22&dq=terraform+documentation&ots=ke_GrBVGzn&sig=oliZvHCcabvPw8jVP26kDLXeXX8 (accessed May 22, 2026).

Appendix

Appendix A: Source Code and User manual

The complete source code used for the implementation of this project is maintained in a public GitHub repository. The repository contains configuration files, scripts, and documentation related to the monitoring setup.

The source code can be accessed at:
https://github.com/githubabhay2003/BTech-Major-Project-Cloud-Native-Observability-EKS

The documentation (user manual) of the project can be accessed at:
https://github.com/githubabhay2003/BTech-Major-Project-Cloud-Native-Observability-EKS/tree/main/docs

Appendix B: GitHub Actions Workflow (deploy.yml)

```
name: Deploy to EKS

on:
  push:
    branches:
      - main

jobs:
  deploy:
    runs-on: ubuntu-latest
    permissions:
      id-token: write
      contents: read
    env:
      AWS_REGION: us-east-1
      IMAGE_TAG: ${{ github.sha }}
    steps:
      - uses: actions/checkout@v3
      - name: Configure AWS credentials (OIDC)
        uses: aws-actions/configure-aws-credentials@v4
        with:
          role-to-assume: arn:aws:iam::ACCOUNT_ID:role/eks-observability-github-actions-role
          aws-region: us-east-1
      - name: Login to ECR
        uses: aws-actions/amazon-ecr-login@v2
      - name: Build and push FastAPI image
        run: |
          docker build -t fastapi-app ./apps/fastapi
          docker tag fastapi-app:latest ACCOUNT_ID.dkr.ecr.us-east-1.amazonaws.com/eks-observability-fastapi:$IMAGE_TAG
          docker push ACCOUNT_ID.dkr.ecr.us-east-1.amazonaws.com/eks-observability-fastapi:$IMAGE_TAG
      - name: Build and push Website image
```

```
      run: |
        docker build -t website ./apps/website
        docker tag website:latest ACCOUNT_ID.dkr.ecr.us-east-1.amazonaws.com/eks-observability-website:$IMAGE_TAG
        docker push ACCOUNT_ID.dkr.ecr.us-east-1.amazonaws.com/eks-observability-website:$IMAGE_TAG
    - name: Update kubeconfig
      run: aws eks update-kubeconfig --region us-east-1 --name eks-observability-cluster
    - uses: azure/setup-helm@v4
    - name: Deploy FastAPI
      run: |
        helm upgrade --install fastapi-app ./helm/fastapi \
          --namespace default \
          --set image.repository=ACCOUNT_ID.dkr.ecr.us-east-1.amazonaws.com/eks-observability-fastapi \
          --set image.tag=$IMAGE_TAG
    - name: Deploy Website
      run: |
        helm upgrade --install website ./helm/website \
          --namespace default \
          --set image.repository=ACCOUNT_ID.dkr.ecr.us-east-1.amazonaws.com/eks-observability-website \
          --set image.tag=$IMAGE_TAG
```

Appendix C: FastAPI Application Source (main.py)

```
from fastapi import FastAPI
from prometheus_fastapi_instrumentator import Instrumentator

app = FastAPI()

@app.get("/")
def root():
   return {"message": "FastAPI Observability App"}

@app.get("/health")
def health():
   return {"status": "ok"}

Instrumentator().instrument(app).expose(app)
```

Appendix D: Grafana Golden Signals Dashboard Queries

Panel Title	PromQL Query
Request Rate (RPS)	sum(rate(http_requests_total[1m]))
Error Rate (5xx/s)	sum(rate(http_requests_total{status=~"5.."}[1m]))

Latency P95	histogram_quantile(0.95, sum(rate(http_request_duration_seconds_bucket[5m])) by (le))
Latency P99	histogram_quantile(0.99, sum(rate(http_request_duration_seconds_bucket[5m])) by (le))
CPU Usage	sum(rate(container_cpu_usage_seconds_total{container="fastapi"}[5m]))
Memory Usage (MB)	sum(container_memory_working_set_bytes{container="fastapi"}) / 1024 / 1024

Appendix E: Terraform Resource Inventory

Module	Resource Type	Resource Name / Description
network	aws_vpc	eks-observability-vpc (10.0.0.0/16)
network	aws_subnet (x4)	2 public + 2 private subnets across 2 AZs
network	aws_internet_gateway	eks-observability-igw
network	aws_nat_gateway	eks-observability-nat (single AZ)
network	aws_eip	Elastic IP for NAT Gateway
network	aws_route_table (x2)	Public and private route tables
network	aws_route_table_association (x4)	Subnet-to-route-table bindings
eks	aws_eks_cluster	eks-observability-cluster (Kubernetes 1.29)
eks	aws_eks_node_group	eks-observability-node-group (m7i-flex.large)
iam	aws_iam_role (x2)	EKS cluster role, EKS node role
iam	aws_iam_role_policy_attachment (x4)	Policy attachments for cluster and node roles
iam	aws_iam_role (GitHub Actions)	eks-observability-github-actions-role
bastion	aws_iam_role	eks-observability-bastion-role
bastion	aws_instance	t3.micro Bastion Host in public subnet
bastion	aws_security_group	SSH ingress on port 22
bastion	aws_key_pair	SSH public key for Bastion access

ecr	aws_ecr_repository (x2)	eks-observability-fastapi, eks-observability-website
apps/observability	helm_release	kube-prometheus-stack (v81.2.2)
apps/observability	helm_release	prometheus-blackbox-exporter
apps/observability	kubernetes_namespace_v1	monitoring namespace
apps/observability	kubernetes_manifest (ServiceMonitor)	fastapi ServiceMonitor
apps/observability	kubernetes_manifest (PrometheusRule x2)	fastapi-alerts, website-alerts
apps/observability	kubernetes_manifest (Probe)	website HTTP probe
apps/observability	kubernetes_config_map_v1	Grafana FastAPI Golden Signals dashboard
apps/ingress-nginx	helm_release	nginx-ingress (LoadBalancer type)
apps/observability-routing	kubernetes_service_v1 (x3)	ExternalName proxies for Grafana, Prometheus, Alertmanager
apps/observability-routing	kubernetes_ingress_v1 (x2)	Observability ingress routing for monitoring tools
apps (root)	kubernetes_config_map_v1_data	aws-auth ConfigMap with RBAC mappings
infra (root)	aws_iam_openid_connect_provider	GitHub OIDC provider

Appendix F: Additional Screenshots

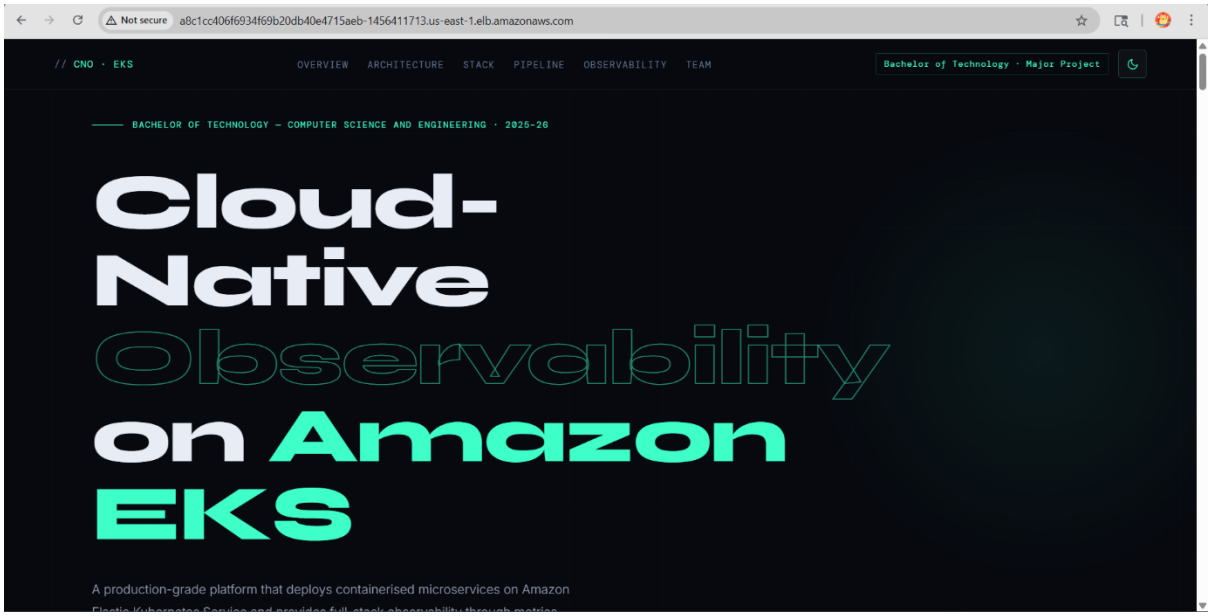

Figure 6: Frontend application exposed via Kubernetes Ingress (public ALB URL)

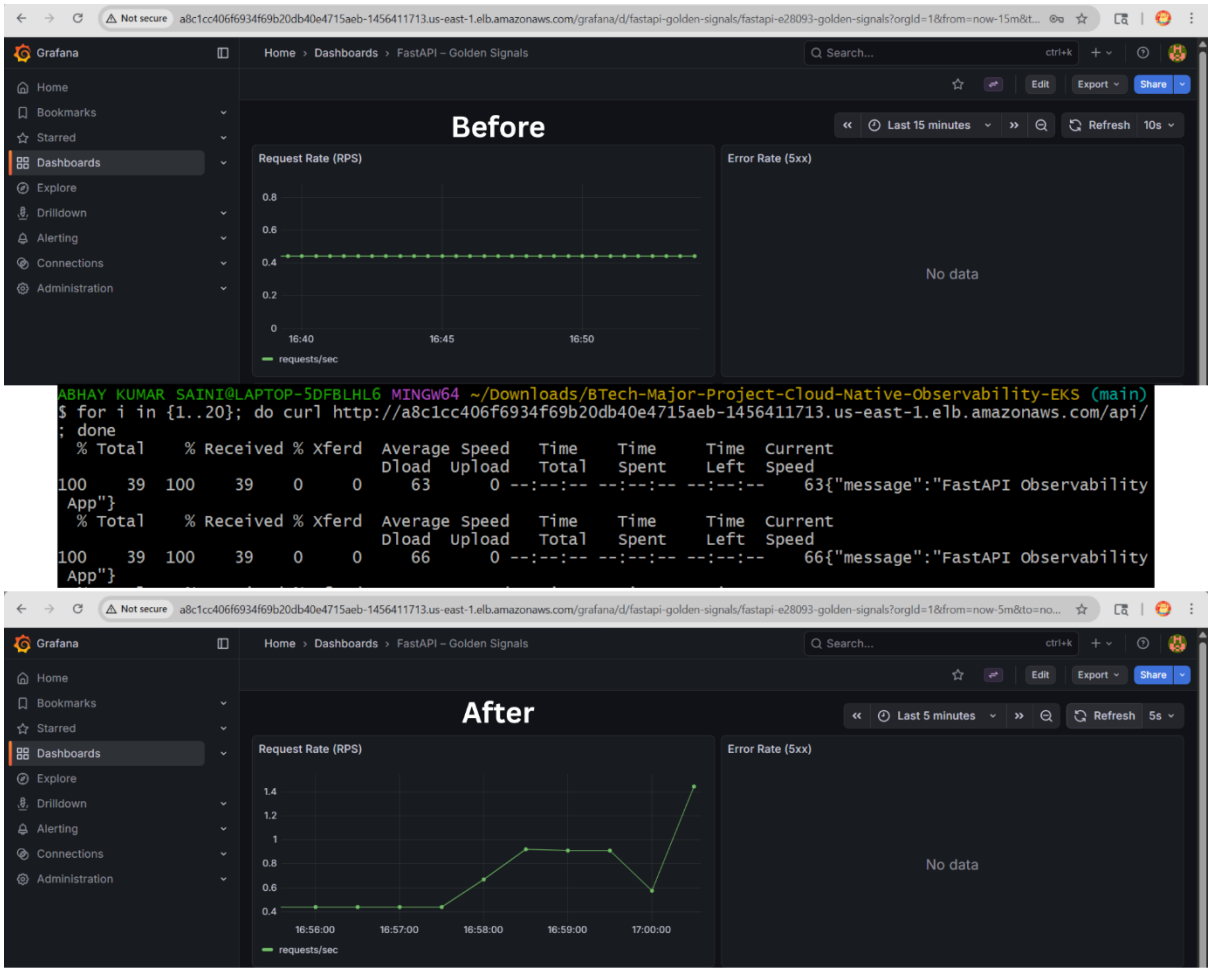

Figure 7: Traffic generation directly reflected in metrics, validating observability pipeline

Figure 8: Grafana dashboard visualizing FastAPI Golden Signals — request rate, latency (P95/P99), error rate, and resource usage (CPU & memory) in real time

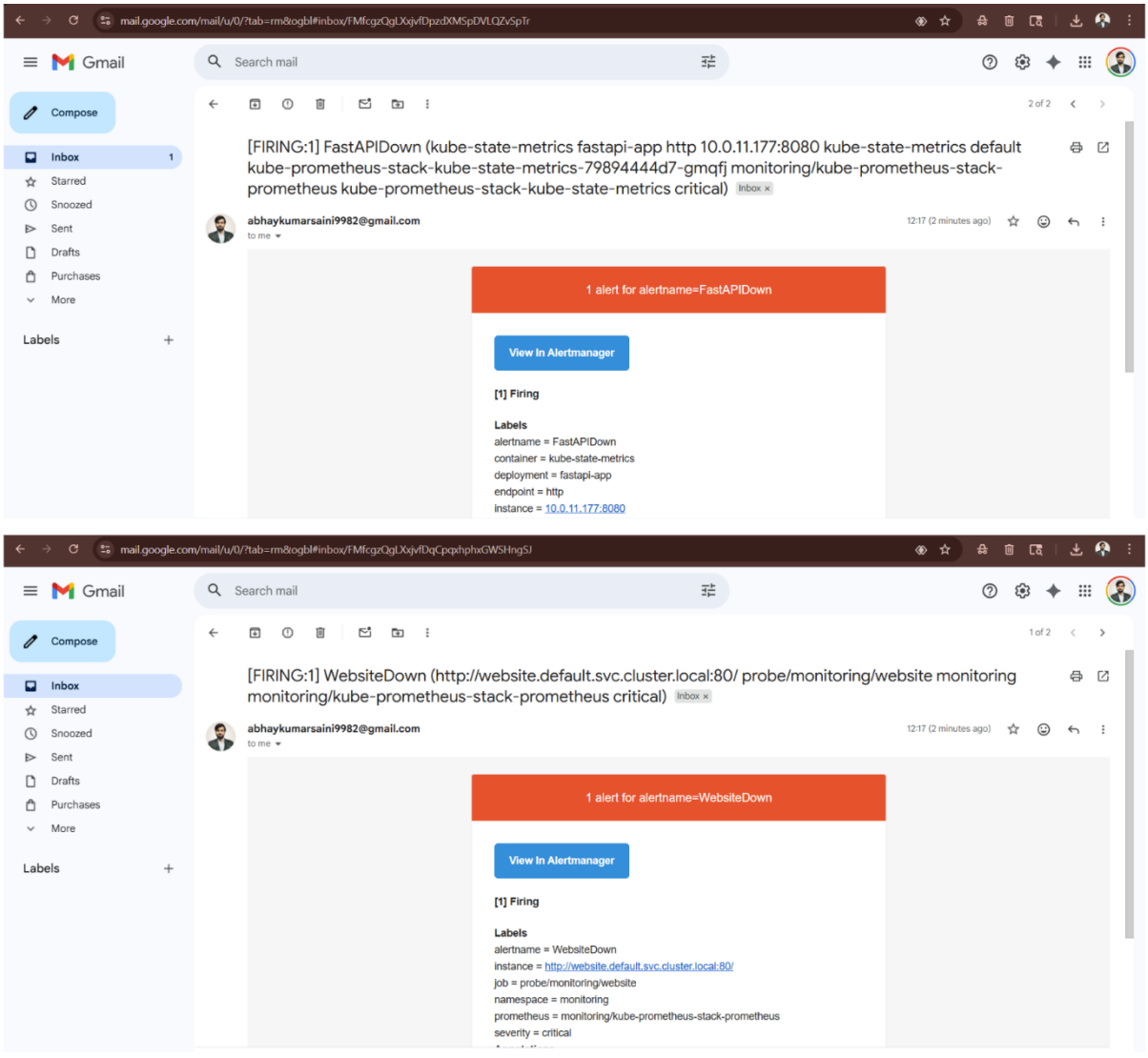

Figure 9: Real-time alert triggered and delivered via email when system thresholds are breached